Inheritance in Java

Inheritance is one of the most important feature of object oriented programming because it allows the creation of hierarchical classification.

A class that is inherited is called a Superclass and the class inheriting is called Subclass.

For inheritance we use extends keyword.

Example :

```
class A
{   int i,j;
    void showA()
    {   SOP(" i and j :" + i +" " + j );
    }
}
class B extends A
{   int k;
    void showB()
    {   SOP(" k :" +k);
    }
    void sum()
    {   SOP(" i+j+k:" + (i+j+k));
    }
}
```

(2)

```
class InheritanceDemo
{
    PSVM (String args[])
    {
        A superOb = new A();
        B subOb = new B();

        superOb.i = 10;
        superob.j = 20;
        superob.showA(); // i and j : 10 20
        subOb.i = 7;
        subOb.j = 8;
        subOb.k = 9;
        subOb.showA(); // i and j : 7 8
        subOb.showB(); // k : 9
        SOP(" Sum of i, j and k in subOb:");
        subOb.sum(); // Sum of i, j and k in subOb
                     // i+j+k : 24
    }
}
```

Although a subclass includes all of the members of its super class, it cannot access private members of super class.

A class member that has been declared as private will remain private to its class. It is not accessible by any code outside the class including subclass.

A superclass variable can reference a subclass object. It is important to understand that it is the type of reference variable determines what members can be accessed, not the type of object it refers to.

Example :

```
class Box
{
    double width;
    double height;
    double depth;
  ✓ Box() { }
  ✓ Box(double w, double h, double d)
    {
        width = w;
        height = h;
        depth = d;
    }
    double volume()
    { return width * height * depth;
    }
}
class BoxWeight extends Box
{
    double weight;
    BoxWeight(double w, double h, double d, double m)
    {
        width = w;
        height = h,
        depth = d;
        weight = m;
    }
}
class RefEnample
{   PSVM(String args[])                    // derived
    {
      ✓ BoxWeight wb = new BoxWeight(3,5,7,8.37);
      ✓ Box pb = new Box(); // Base
        double vol;
        vol = wb.volume();
```

(1)
```
    sop("volume of weightbox is " + vol);   (3*5*7)
    pb = wb;
    vol = pb.volume();
    sop("volume of plainbox is " + vol);
    // sop("weight of plainbox is " + pb.weight);
    }
}
```

In this example plainbox can not access to weight even if it refers to a Boxweight object.

Super keyword:

Super keyword differentiates base class features from derived class.

Super keyword plays important role in three places:
- (a) at variable level ✓
- (b) at method level ✓
- (c) at constructor level ✓

(a) Super at variable level:

When we inherite the base class data member into derived class there is a possibility that base class members and derived class member are same. This situation leads to ambiguity.

To differentiate the base class members and derived class members at derived class level, the base class members must be preceded with super keyword.

Example :

```
class Base
{
    int a;
}

class Derived extends Base
{
    int a;
    int c;

    void set(int x, int y)
    {
        super.a = x;
        this.a = y;
    }
    void add()
    {
        c = super.a + a;
    }
    void display()
    {
        sop("Value of a from Base =" + super.a);
        sop("Value of a from Derived =" + a);
        sop("Sum =" + c);
    }
}

class Test
{
    psvm(-)
    {
        Derived d = new Derived();
        d.set(10,20);
        d.add();
        d.display();
    }
}
```

(b)

(b) Super at constructor level :

A subclass can call a constructor defined by superclass. The general form of super is :

super (argument if any)

It is important to note that super() must be the first statement inside subclass constructor.

Example :

```
class Box      // Base
{
    private double width;
    private double height;
    private double depth;

  ✓ Box(Box ob)
   {   width = ob.width;
       height = ob.height;
       depth = ob.depth;
   }
   Box (double w, double h, double d)
   {
       width = w;
       height = h;
       depth = d;
   }
   Box()
   {   width = -1;
       height = -1;
       depth = -1;
   }
   Box (double len) { width = height = depth = len; }
```

```
    double volume()
    { return width * height * depth;
    }
}

class BoxWeight extends Box  // Derived class.
{   double weight;
    BoxWeight (BoxWeight ob)
    { super(ob);
      weight = ob.weight;
    }
    BoxWeight (double w, double h, double d, double m)
    { super(w, h, d);
      weight = m;
    }
    BoxWeight()
    { super();
      weight = -1;
    }
    BoxWeight (double len, double m)
    { super(len);
      weight = m;
    }
}

class SuperDemo
{
    PSVM(-)
    {
        BoxWeight bw1 = new BoxWeight(10,20,15,34.3);
        BoxWeight bw2 = new BoxWeight(2,3,4,0.076);
        BoxWeight bw3 = new BoxWeight();
        BoxWeight bc1 = new BoxWeight(3,2);
        BoxWeight bc2 = new BoxWeight(bw1);
        double vol;
```

(4)

```
vol = bw1.volume();
sop("volume of bw1 is =" + vol);
sop("weight of bw1 is =" + bw.weight);
vol = bw2.volume();
sop("volume of bw2 is" + vol);
sop("weight of bw2 is " + bw2.weight);

vol = bw3.volume();
sop("volume of bw3 is" + vol);
sop("weight of bw3 is" + bw3.weight);

vol = bc1.volume();
sop("volume of clone is" + vol);
sop("weight of clone is", bc1.weight);
vol = bc2.volume();
sop("volume of cube is" + vol);
sop("weight of cube is" + bc2.weight);

    }
}
```

Output :

```
volume of bw1 is 3000.0
weight of bw1 is 34.3
volume of bw2 is 24.0
weight of bw2 is 0.076
volume of bw3 is -1.0
weight of bw3 is -1.0
volume of clone is 3000
weight of clone is 34.3
volume of cube is 27.0
weight of cube is 2.0
```

How constructors are executed : :

```
class A
{
    A()
    {
        SOP("Inside A's constructor");
    }
}

class B extends A
{
    B()
    {
        SOP("Inside B's constructor");
    }
}

class C extends B
{
    C()
    {
        SOP("Inside C's constructor");
    }
}

class ConstructorExecution
{
    PSVM(-)
    {
        C c = new C();
    }
}
```

Output :

Inside A's constructor ✓
Inside B's constructor ✓
Inside C's constructor ✓

(10)

Method overriding :

In inheritance when a method in subclass has the same name and signature as in its superclass the method in subclass is said to be overriden in the superclass.

When an overridden method is called from its subclass, it always refers to the version defined by the subclass.

Example :

```
class A
{   int i,j;
    A(int a, int b)
    {   i = a;
        j = b;
    }
    void show()
    {   sop("i and j :" + i + " " + j);
    }
}

class B extends A
{   int k;
    B(int a, int b, int c)
    {   super(a,b);
        k = c;
    }
    void display
    void show()
    {   super.show() // calls super class show()
        sop("k:" + k);
    }
}
```

```
class override
{
    PSVM(-)
    {
        B b = new B(1, 2, 3);
        b.show();
    }
}
```

output: i and j : 1 2
K : 3

Dynamic Method Dispatch :

Dynamic method dispatch is a mechanism by which a call to an overridden method is resolved at runtime.

By dynamic method dispatch Java runtime polymorphism is implimented.

A superclass ~~object~~ reference variable can refer to a subclass object.

When an overriden method is called through a superclass reference, JAVA determines which version of that method to execute based upon the type of object being refered to at the time of call.

This determination is made at runtime.

It is the type of object being reffered to (not the type of reference variable) that determines which version of overridden method will be executed.

Example :

```
class A
{   void callme()
    {   sop("Inside A's callme");
    }
}

class B extends A
{   void callme()
    {   sop("Inside B's callme");
    }
}

class C extends B
{   void callme()
    {   sop("Inside C's callme");
    }
}

class DMD
{   PSVM(-)
    {   A a = new A();
        B b = new B();
        C c = new C();

        A r;
        r = a;
        r.callme();
        r = b;
        r.callme();
        r = c;
        r.callme();
    }
}
```

Output

```
Inside A's callme method
Inside B's callme method
Inside C's callme method
```

Example:

```
class Figure
{
    double dim1;
    double dim2;
    Figure (double a, double b) {
        dim1 = a; dim2 = b;
    }
    double area() {
        Sop("Area of Figure is undefined.");
        return 0;
    }
}

class Rectangle extends Figure
{
    Rectangle(double a, double b)
    { super(a,b);
    }
    double area() {
        Sop("Inside Area for Rectangle.");
        return dim1 * dim2;
    }
}
class Triangle extends Figure
{
    Triangle (double a, double b)
    { super(a,b);
    }
    double area()
    {
        Sop("Inside Area for Triangle.");
        return dim1 * dim2 / 2;
    }
}
```

(14)

```
class FindAreas {
    PSVM(-) {
    Figure f = new Figure(10, 10);
    Rectangle r = new Rectangle(9, 5);
    Triangle t = new Triangle(10, 8);

    Figure figref;
    figref = r;
    SOP("Area is" + figref.area());
    figref = t;
    SOP("Area is" + figref.area());
    figref = f;
    SOP("Area is" + figref.area());
}
}
```

Abstract classes :

There are situations in which you will provide a super class with a structure of a given abstraction without providing a complete implimentation of every method.

In previous example Figure class, definition of area() is simply a place holder. It will not compute and display the area of any type of object.

If you want certain method to be overridden by subclass, you have to specify the method by "abstract" type modifier.

the general form is

abstract type-name (parameter-list);

with no method body.

Any class that contains one or more abstract method must be declared abstract.

To declare a class abstract, you simply use the abstract keyword infront of class keyword.

Objects of abstract class can not be created. An abstract class cannot be instantiated by using new operator.

Imp

Example :

```
abstract class A
{
    abstract void callme();
    void callmetoo()
    { sop("This is a concrete method");
    }
}

class B extends A
{
    void callme()
    { sop("B's implementation to callme()");
    }
}

class AbstractDemo
{
    PSVM(-){
        B b = new B();
        b.callme();
        b.callmetoo();
    }
}
```

Example :-

```
abstract class Figure {
        double dim1;
        double dim2;
    Figure(double a, double b) {
            dim1 = a;
            dim2 = b;
        }
    abstract double area();
}

class Rectangle extends Figure {
    Rectangle(double a, double b) {
        super(a, b);
    }
    double area() {
        SOP("Inside area for Rectangle.");
        return dim1 * dim2;
    }
}

class Triangle extends Figure {
    Triangle(double a, double b) {
        super(a,b);
    }
    double area() {
        SOP("Inside area for Triangle.");
        return dim1 * dim2/2;
    }
}

class AbstractAreas {
    PSVM(-) {
        // Figure f = new Figure(10,10); // Illegal
        Rectangle r = new Rectangle(9,5);
        Triangle t = new Triangle(10,8);
        Figure figref;
        figref = r;
        SOP("Area is" + figref.area());
        figref = t
        SOP("Area is" + figref.area());
    }
}
```

final with Inheritance :

methods declared final cannot be overridden.

```
class A {
    final void method1() {
        Sop ("This is a final method");
    }
}

class B extends A {
    void method1() {        // cannot override.
        Sop ("Illegal");
    }
}
```

So final method prevents method overriding.

A class declared with final cannot be inherited. Declaring a class final implicitly declares all its methods final.

So it is illegal to declare a class as both final and abstract.

— o —